SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

Assessment Tool 2: Short Answer Test

Course Code	**CSA3717**	**Course Title**	**Software testing**
CO Assessed	**CO1**	**Assessment**	**Assessment Tool 2: Short Answer Test**
Weightage	**35% of CIA**	**Total Marks**	**100**
CO1 Statement	***Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity.* (BL2, BL3)**		
Student Name	**B.Thejaswini**	**Reg. No.**	**192412479**
Section/Batch	**Slot A /**	**Date**	**23-10-2026**

Code of Conduct:

I B.Thejaswini, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

B.Thejaswini

SIMATS ENGINEERING

Short Answer Test - Software Testing (CSA3717)

CO1 - Answers to Questions 1 to 15

Q1. A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.

Answer:

Testing is an engineering activity, which means it must be planned, designed, executed and measured just like any other engineering work, rather than done by guesswork. Random testing depends on individual intuition, so coverage cannot be measured, results cannot be repeated and many defects are missed.

To redesign the approach, I would first study the requirements and define the test objectives and scope. Next, I would prepare a test plan with entry and exit criteria, and then apply systematic design techniques such as equivalence partitioning, boundary value analysis and decision tables to derive test cases. Each test case would have an ID, input, preconditions, expected result and a link to a requirement (a traceability matrix). Finally, tests would be executed, defects logged in a tracking tool, and metrics such as coverage and defect density reviewed.

Structured test design improves quality in several ways. It gives systematic coverage of all requirements, so no feature is skipped. Techniques target the places where defects usually hide, which raises defect detection. Test cases are repeatable and reusable, so results are consistent across testers and builds. Redundant tests are removed, saving time, and progress can be measured objectively.

Example: for an age field accepting 18 to 60, random testing may try only 25 or 30. Structured design would test 17, 18, 19, 59, 60, 61, which are the values where defects typically occur. Thus structured design makes the startup's testing consistent, measurable and far more effective.

Q2. A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.

Answer:

A defined testing process is a documented, repeatable set of activities: test planning, test design, test environment setup, test execution, defect reporting and tracking, and test closure. In a banking system, where a single failure can mean financial loss and loss of customer trust, such discipline is essential.

Illustration: when a funds-transfer feature is developed, planning identifies it as a high-risk module. Test cases are designed for valid transfers, insufficient balance, daily limit exceeded, invalid account numbers, network failure mid-transaction and simultaneous transfers. These

are executed in a controlled environment with test data, defects are logged and retested, and exit criteria such as zero critical defects must be met before release.

Benefits of a well-defined process:

- Consistency: every module is tested the same way, so quality is uniform.
- Early defect detection: reviews and planning catch issues before they become costly production failures.
- Risk reduction: critical areas such as payments, balance updates and security get priority.
- Traceability and accountability: each requirement maps to test cases and results, which also helps with audits and compliance.
- Reliability: concurrency, rollback and recovery scenarios are tested, preventing issues like double debit.

Example: a defined process would include a test for a transaction that fails halfway, ensuring the amount is refunded or rolled back. Without a process this scenario is easily forgotten. Hence, standardizing testing improves quality, reliability and customer confidence.

Q3. During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.

Answer:

Testing should run in parallel with every SDLC phase, as in the V-model, rather than only at the end. Each development phase has a matching test activity.

- Requirement analysis: testers review requirements for clarity and testability and start preparing the test plan and acceptance criteria.
- Design: system and integration test cases are designed; design documents are reviewed.
- Coding: developers perform unit testing and code reviews on modules such as cart, search and payment.
- Integration testing: modules are combined and interfaces are checked, for example cart to payment gateway to order confirmation.
- System testing: the complete platform is tested for functionality, performance, security and compatibility.
- Acceptance testing: users or business stakeholders validate the platform against their needs before release.
- Maintenance: regression testing is done after every change or fix.

Process discipline ensures better quality because defects are caught in the phase where they are introduced, which is cheaper to fix. Integration and system defects reduce because interfaces are tested systematically and not assumed to work. Clear entry and exit criteria prevent half-tested software from moving forward. Documentation and traceability make results repeatable and measurable.

Example: if the payment interface is tested at integration level, a mismatch in the amount format between the cart and payment modules is found early, instead of causing failed orders in the live system. Thus a structured testing process across the SDLC gives predictable, higher-quality outcomes.

Q4. A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.

Answer:

Definitions: An error (mistake) is a human action that produces an incorrect result, for example a programmer misunderstanding a requirement. A defect (bug or fault) is the flaw in the code or document that results from the error. A failure is the observable incorrect behaviour of the software when the defect is executed.

Classification of the login issue: The mismatch between expected and actual output seen during validation is a failure, because it is the visible deviation in behaviour. For example, the expected result is that a user with valid credentials logs in, but the actual result is an 'Invalid password' message. The cause of this failure is a defect in the code, such as using the wrong comparison condition or ignoring case or special characters in passwords. The root of the defect is an error made by the developer, such as misreading the password rule.

Relationship: Error → Defect → Failure. A human error introduces a defect into the software; when the defective code is executed under certain conditions, it causes a failure. Not every defect causes a failure: if the faulty code is never executed, or is executed with inputs that do not trigger it, no failure is seen. Also, failures can arise from environmental conditions, not only defects.

Another example: a developer mistakenly writes if (attempts > 3) instead of if (attempts >= 3) (error). This is a defect in the lockout logic. The user is locked out only after four attempts instead of three, which is the failure. Hence the login problem observed is a failure, caused by an underlying defect that was introduced by a developer's error.

Q5. A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.

Answer:

The principle that applies here is 'Early testing' (shift-left testing). It states that testing activities should start as early as possible in the software development life cycle and be focused on defined objectives.

The cost of fixing a defect rises sharply the later it is found. A requirement defect found during requirement review may cost almost nothing to fix, while the same defect found in production may require redesign, recoding, retesting and redeployment, along with customer impact.

Applying the principle: testers join requirement and design reviews to find ambiguities and gaps; test cases are prepared while requirements are being finalized; developers perform unit testing and static analysis or code reviews; automated tests are run through continuous integration on every build so that problems are reported immediately.

How it improves effectiveness: defects are found in the phase where they are introduced, so rework is small. Requirement defects do not spread into design and code. Testers understand

the system better and write better test cases. Feedback to developers is quick, and the project cost, effort and schedule risk reduce.

Example: in a shopping application, the requirement says 'discount should be applied on large orders' without defining 'large'. A tester reviewing the requirement raises the question early, and it is clarified as 'orders above Rs. 5000'. If this had been found only in system testing, the logic, code and test cases would all have needed rework. Thus early testing improves defect detection efficiency and lowers cost.

Q6. During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.

Answer:

A tester contributes to the requirement phase through static testing, which means examining documents without executing code. Typical contributions include:

- Reviewing and inspecting the requirement specification to find ambiguity, contradictions, missing details and untestable statements.
- Asking clarifying questions, such as what 'quickly retrieve patient records' means in seconds.
- Checking that every requirement is clear, complete, consistent, measurable and testable.
- Identifying missing non-functional requirements such as data privacy, security, audit logs and response time.
- Preparing acceptance criteria, early test scenarios and a requirement traceability matrix.
- Participating in risk analysis to identify high-risk areas such as medication dosage and patient identity.

Example: a requirement says 'The system shall alert the doctor for abnormal values'. A tester asks: what counts as abnormal, who sets the limit, and how is the alert delivered? Resolving this prevents wrong implementation of alerts.

Importance of early involvement: defects introduced in requirements multiply through design, code and testing, so fixing them later is costly. In healthcare, a defect may even endanger patient safety. Early involvement improves the clarity of requirements, reduces rework, shortens the schedule, builds shared understanding between business, developers and testers, and makes later test design faster and more accurate. Therefore the tester's early participation is a key quality-improvement activity.

Q7. A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.

Answer:

Defects originate from human errors made at any stage of the lifecycle, and studies show that a large share of them originate in the requirement and design phases.

How defects originate in the payroll system:

- Requirement phase: ambiguous statements such as 'calculate salary with applicable deductions' do not specify tax slabs, PF rates, overtime or leave rules. Different people interpret them differently.
- Design phase: the designer builds the logic on an assumption, for example rounding off at each step rather than at the final amount.
- Coding phase: the developer implements the assumed logic, and coding mistakes add further defects, such as wrong formula or data type.
- Testing phase: testers derive expected results from the same unclear requirements, so the defects remain hidden and reach production.

This shows a defect propagation: an ambiguity in the requirement becomes a flaw in design, then in code, and finally appears as a wrong salary slip (failure).

How requirement clarity prevents such issues: clear, complete, consistent and testable requirements, with precise formulas, rules, examples and boundary conditions, leave no room for assumptions. Requirement reviews and walkthroughs involving business users, developers and testers detect ambiguity early. Use of examples, for instance 'Basic 30,000 with 12% PF gives PF of 3,600', provides a common understanding. Traceability then ensures every requirement is designed, coded and tested.

Therefore, improving requirement clarity removes the source of defects, and reduces rework, cost and incorrect payroll outputs.

Q8. A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.

Answer:

Classification: The defect can be classified as a functional / input validation (boundary condition) defect, with a severity of high or critical since it causes a system crash. It is typically caused by a coding error such as using '<' instead of '<=', off-by-one errors, or missing validation of limits. It is not a performance or usability defect, because the problem is incorrect handling of specific input values, not speed or ease of use.

Justification: The system works for normal values but fails exactly at the limits of the allowed input range. This pattern is the characteristic sign of a boundary defect. The failure indicates that the test coverage ignored edge values and focused only on typical inputs.

How boundary value analysis (BVA) could have prevented it: BVA is a black-box test design technique based on the fact that defects occur more often at the boundaries of input ranges than at the centre. For each range, test cases are chosen at the minimum, just above the minimum, a nominal value, just below the maximum, and the maximum, and also just outside the limits (min-1 and max+1).

Example: if a field accepts quantity from 1 to 100, BVA would test 0, 1, 2, 50, 99, 100 and 101. Testing 0 and 101 would reveal whether invalid values are rejected properly, and testing 1 and 100 would confirm the limits are accepted. If the code wrongly used 'quantity < 100', the value 100 would fail and the defect would be caught in testing.

Thus applying BVA would have improved coverage at the edges, exposed the crash before release, and prevented the failure.

Q9. A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.

Answer:

A defect repository is a central database that stores details of all defects found, such as ID, description, module, severity, priority, root cause, phase of origin, status, and fix. When analysed regularly, it becomes a valuable knowledge base rather than just a record.

How it supports better test case design:

- Identifying defect-prone modules: modules with many defects can be given more test cases and deeper testing.
- Learning from past failures: for every defect found earlier, a new test case is added so the same defect is checked in future releases.
- Discovering defect patterns: if most defects are boundary or validation errors, test design can emphasise BVA and negative testing.
- Root cause analysis: causes such as unclear requirements or missing checks guide process improvement and review checklists.
- Regression suite building: fixed defects are converted into regression tests.

Example: if the repository shows repeated defects in date-format handling in the billing module, testers add test cases for different formats, leap years and month ends, and run them in every release.

Role in improving software quality: the repository helps prevent recurring defects, focus effort where risk is highest, track defect trends and metrics such as defect density and leakage, support decision making on release readiness, and improve communication between testers and developers. Using it actively changes testing from reactive to preventive and reduces repeated defects across releases.

Q10. An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.

Answer:

A functional defect is a flaw because of which the software does not behave according to its specified functional requirements. In the ATM, the withdrawal feature is supposed to check the balance, dispense the correct cash, deduct the amount exactly once and update the account. Failure due to incorrect transaction logic means the feature does not behave as specified, so it is a functional defect.

Description: For example, the account balance may be deducted but cash not dispensed, the amount may be deducted twice, withdrawal may be allowed beyond the balance or daily limit, or the transaction may not roll back when the network or the cash dispenser fails. Such defects are usually caused by errors in business logic, missing conditions or improper handling of exceptions. Because they affect money, they are high severity.

How it could have been prevented:

- Requirement review to ensure all withdrawal rules and error scenarios are clearly defined.

- Test design using equivalence partitioning and boundary value analysis (balance equal to the withdrawal amount, amount just above the limit) and decision tables for combinations of conditions.
- Negative and exception testing, such as power or network failure in the middle of a transaction, and card retained or cash jam.
- State transition and transaction testing to verify rollback and consistency of data.
- Code reviews and unit testing of the transaction logic, followed by integration, system and regression testing.

Applying these practices would have exposed the defect before release and prevented the failure in the live ATM.

Q11. A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.

Answer:

Defect clustering is a software testing principle which states that a small number of modules usually contain most of the defects, which follows the Pareto principle (roughly 80% of defects come from 20% of the modules). The causes include complex logic, frequent changes, tight deadlines, new technology, inexperienced developers or many interfaces.

Identifying testing priorities using defect clustering:

- Collect defect data from earlier cycles and the defect repository, and group it by module.
- Rank the modules by defect count, severity and change frequency to find the defect-prone ones (hot spots).
- Allocate more testing effort, more test cases, deeper techniques and experienced testers to these modules.
- Run regression tests on them more frequently and review their code and design in detail.
- Keep basic coverage for stable, low-risk modules so they are not ignored.
- Keep updating the ranking, because clusters may shift over time.

How it improves efficiency: limited time and resources are used where the chance of finding defects is highest, so more defects are found in less effort. Critical defects are detected earlier, risk is reduced, and unnecessary testing of stable modules is avoided. It also gives developers guidance for refactoring problem modules.

Example: in an application with 10 modules, if the payment and order modules produce 70% of defects, the team assigns most test effort to them. This results in better defect detection and an efficient use of the testing budget.

Q12. A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.

Answer:

Defect classification is the process of grouping defects based on attributes such as type, severity, priority, phase of origin and cause. Common defect types are functional, performance, usability, security, compatibility, interface and data defects.

Classification of the given issue:

- Type: a non-functional defect, specifically a performance defect (scalability or response-time problem), because the features work correctly but the system is slow under load.
- Detected by: performance, load and stress testing.
- Severity: high, because users face delays or timeouts at peak times, while functionality is not completely blocked.
- Priority: high, particularly if it affects key transactions such as login or checkout.
- Probable origin: design or coding phase, for example unoptimised database queries, missing indexing, no caching, memory leaks, inefficient algorithms or insufficient server capacity.

How classification helps in better resolution:

- It routes the defect to the right team, such as performance engineers or database administrators, instead of general developers.
- Severity and priority help to schedule the fix correctly.
- It points to suitable tools and techniques such as load testing, profiling and monitoring.
- It supports root cause analysis and trend analysis, showing which defect types are most frequent.
- It helps in preventing similar defects by improving design reviews and performance requirements.

Thus proper classification makes the resolution faster, more focused and more effective.

Q13. A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.

Answer:

Regression testing confirms that new changes or fixes have not broken existing functionality. As the application grows, running all tests every time is expensive, so historical defect data helps to select and prioritise tests wisely.

How historical defect data can be reused:

- Convert defects into test cases: every defect that was found and fixed gets a test case in the regression suite, so it cannot return unnoticed.
- Identify defect-prone modules: modules with many past defects are given higher priority and more regression tests.
- Analyse patterns and root causes: recurring problems such as validation or integration defects lead to targeted test scenarios.
- Prioritise by severity and impact: tests for high severity defects and critical business flows are run first.
- Impact analysis: by linking defects to code areas and changes, tests related to the modified components are selected.
- Remove or reduce tests that have never found any defects and cover stable areas, keeping the suite lean.

How it improves coverage and efficiency: coverage improves because known weak areas and past failure scenarios are always tested. Efficiency improves because only relevant, high-risk tests are executed, so execution time and cost fall. It also supports automation, since stable, repeated defect scenarios are good candidates for automated regression scripts.

Example: if earlier releases showed repeated defects in the discount calculation, those scenarios are included and automated in every regression cycle, which quickly detects any new break in that area.

Q14. During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.

Answer:

Poor communication leads to different interpretations of requirements and interfaces, which result in integration defects such as mismatched data formats, wrong parameters, or missing interface handling. The tester plays an important role in reducing these defects.

Role of the tester:

- Participate in requirement and design reviews and ask questions to clarify assumptions about interfaces and data exchange.
- Share test scenarios and test cases early with developers so both sides have the same understanding of expected behaviour.
- Prepare integration test plans that cover interfaces, data flow, error handling and dependencies among modules.
- Report defects clearly and with evidence (steps, logs, expected versus actual results) so developers can reproduce and fix them quickly.
- Take part in daily meetings, defect triage and retrospectives to maintain regular communication.
- Verify fixes and run regression tests to ensure integration is not broken again.

How early involvement helps: when testers join from the requirement and design stages, ambiguities and interface mismatches are discovered before coding. Developers and testers share one view of the system, so fewer misunderstandings occur. Defects are cheaper to fix, rework is reduced, and the integration phase becomes smoother with fewer surprises.

Example: if a tester finds in a design review that one module sends the date as DD-MM-YYYY while another expects YYYY-MM-DD, the mismatch is fixed immediately, preventing an integration defect later.

Q15. A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.

Answer:

Defect prevention aims to stop defects from being introduced in the first place, instead of only finding and fixing them. Repeated defects show that the root causes have not been addressed, so a long-term, process-oriented solution is required.

Defect prevention techniques that can be applied:

- Root cause analysis (RCA): analyse recurring defects from the repository, using tools such as Pareto charts and fishbone diagrams, to find why they occur.
- Corrective and preventive actions: change the process, tools or training that caused the defect, not just the code.

- Reviews and inspections of requirements, design and code to remove defects early.
- Coding standards, checklists and static code analysis tools to avoid common mistakes.
- Training and knowledge sharing for the team about frequent error types.
- Better requirement and design practices, such as clear, testable requirements and design reviews.
- Automated regression and continuous integration to catch repeated issues immediately.
- Lessons learned and retrospectives after every release.

How prevention improves quality: fewer defects are injected, so the product is more reliable and stable. Rework, cost and time spent on fixing reduce, and release cycles become more predictable. Customer satisfaction improves because fewer defects reach production. The team's process maturity grows, moving from reactive fixing to proactive quality management.

Example: if date-handling defects recur, introducing a common validated date utility, a coding guideline and a review checklist item will prevent the problem in future releases.

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions	
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation	
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis	
Organization	20%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers	
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps	